TUGAS PRAKTIKUM 6

PEMPROGRAMAN MOBILE



Disusun Oleh :

Nama : Salu Rahmawati

NIM :20240801001

Fakultas : Ilmu komputer

PROGRAM STUDI TEKNIK INFORMATIKA

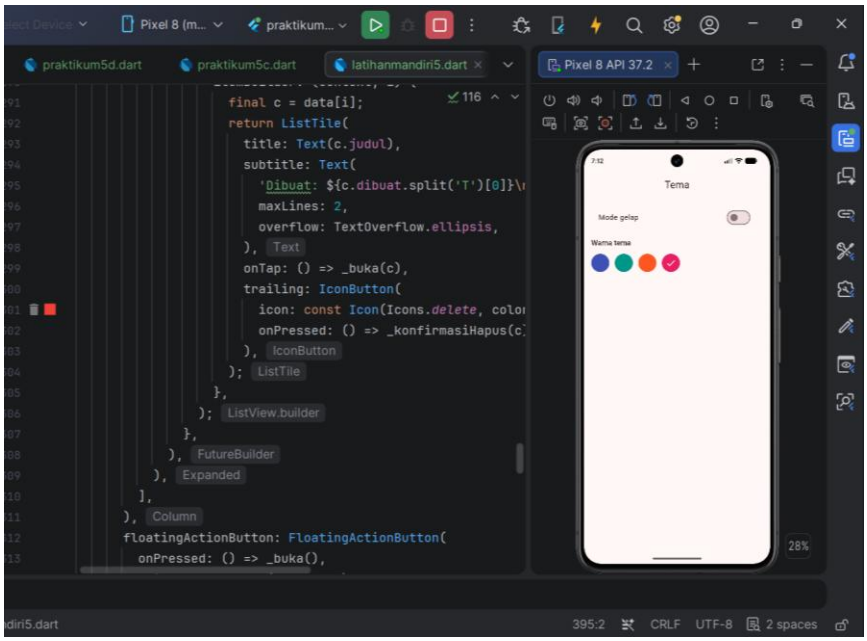
FAKULTAS ILMU KOMPUTER

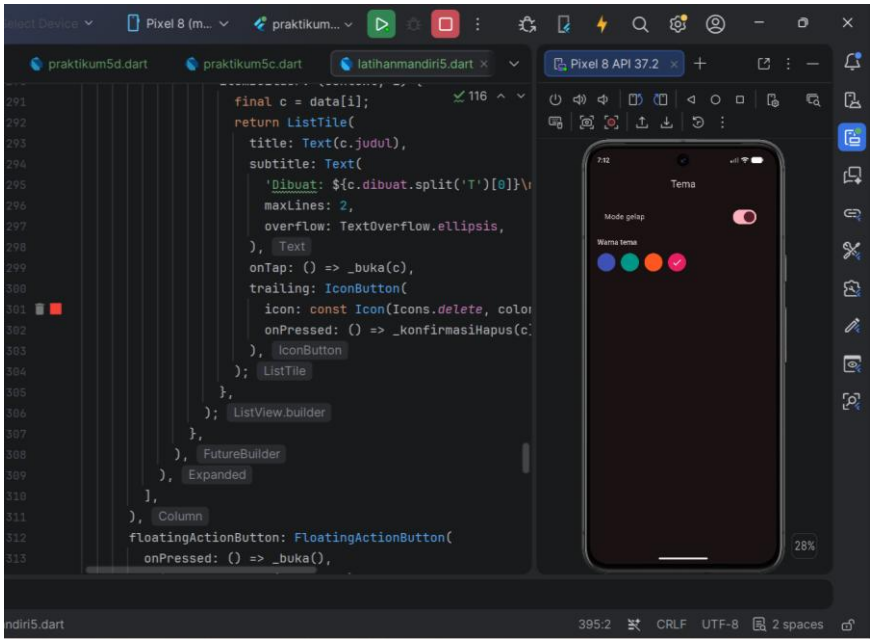
UNIVERSITAS ESA UNGGUL TANGERANG

2026

Bagian A — Tema Terang/Gelap dan Warna Tema

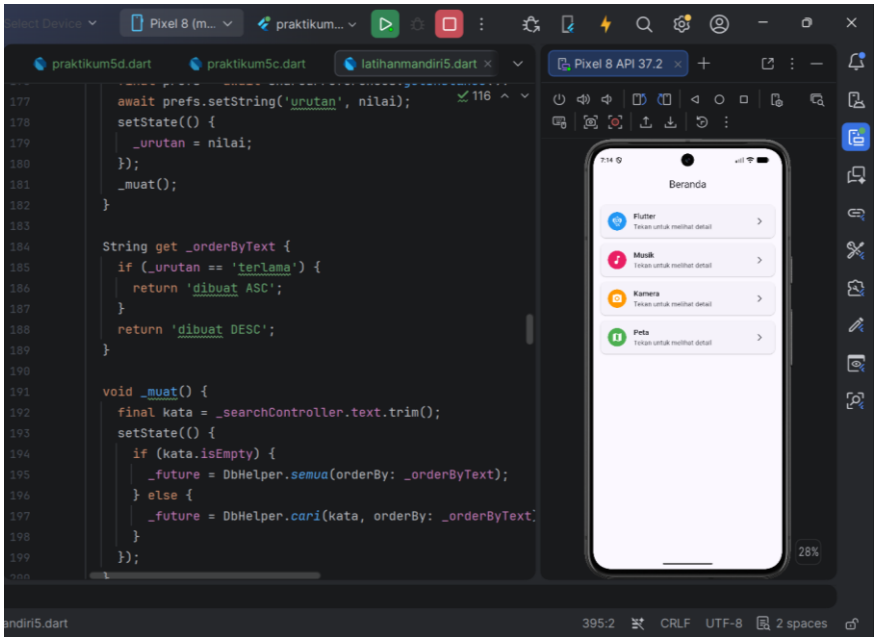
Pada Bagian A praktikum dilakukan penerapan tema terang dan gelap serta perubahan warna tema pada aplikasi Flutter menggunakan ThemeData, ThemeMode, ValueNotifier, dan ListenableBuilder. Aplikasi menyediakan SwitchListTile untuk mengubah mode terang dan gelap serta beberapa pilihan warna yang dapat dipilih oleh pengguna. ThemeData digunakan untuk mengatur tampilan aplikasi dengan colorSchemeSeed sebagai warna dasar, sedangkan theme dan darkTheme digunakan untuk menyediakan tema terang dan gelap. Perubahan tema dapat dilakukan secara langsung tanpa melakukan restart aplikasi sehingga tampilan aplikasi menjadi lebih dinamis dan dapat disesuaikan oleh pengguna.

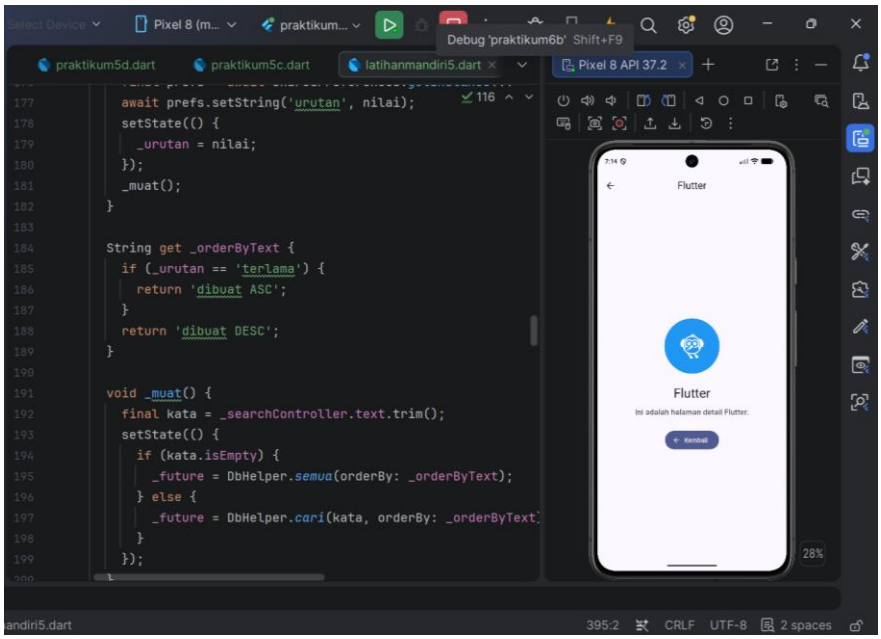




Bagian B — Named Routes, Argumen, dan Hero

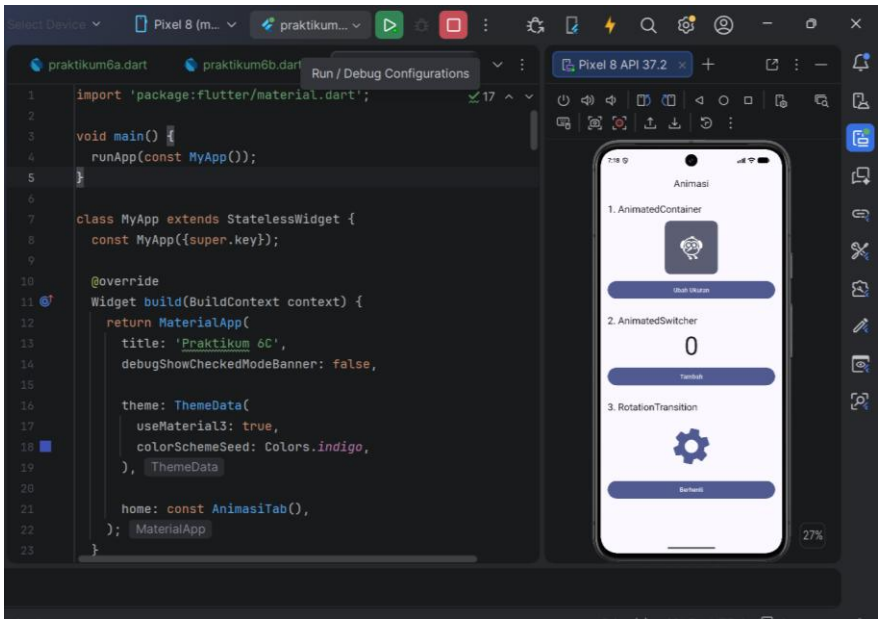
Pada Bagian B diterapkan named routes untuk melakukan navigasi antar halaman secara lebih terstruktur. Data item yang dipilih pada halaman Beranda dikirim ke halaman detail menggunakan Navigator.pushNamed() melalui parameter arguments, kemudian diterima dan diproses menggunakan onGenerateRoute. Selain itu, digunakan widget Hero untuk memberikan efek transisi pada ikon ketika berpindah dari halaman daftar menuju halaman detail. Widget Hero pada kedua halaman menggunakan tag yang sama sehingga Flutter dapat mengenali elemen yang akan dianimasikan. Dengan penerapan ini, navigasi antar halaman menjadi lebih terstruktur dan tampilan aplikasi menjadi lebih interaktif.

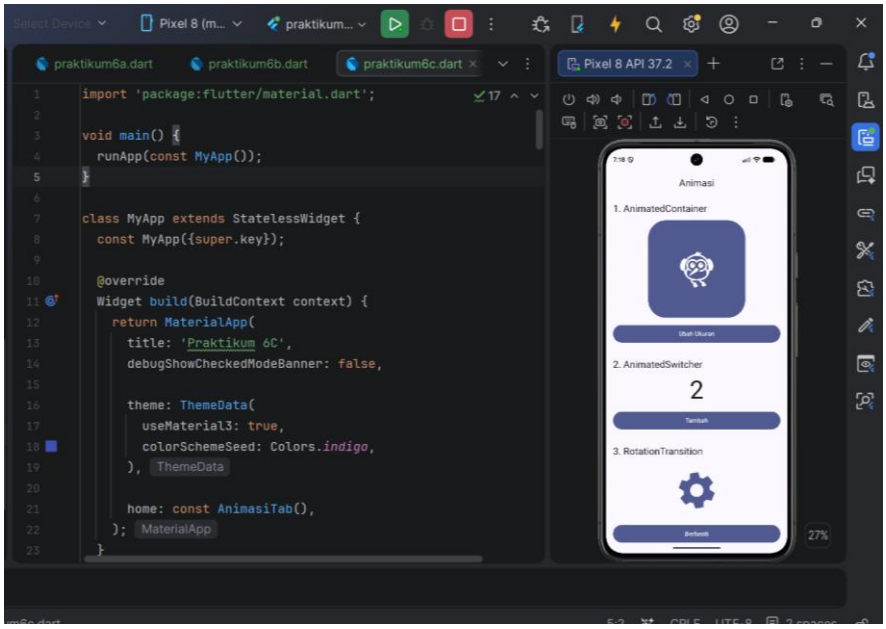




Bagian C — Animasi

Pada Bagian C dilakukan penerapan animasi pada Flutter menggunakan AnimatedContainer, AnimatedSwitcher, dan AnimationController. AnimatedContainer digunakan untuk membuat perubahan ukuran, warna, dan bentuk container secara halus ketika nilai diubah melalui setState. AnimatedSwitcher digunakan untuk memberikan efek transisi ketika nilai penghitung berubah dengan menggunakan ValueKey yang berbeda. Selanjutnya, AnimationController digunakan sebagai animasi eksplisit untuk membuat ikon berputar yang dapat dijalankan, dihentikan, dan diulang kembali. Dengan demikian, pada bagian ini dipelajari perbedaan antara animasi implisit yang lebih sederhana dan animasi eksplisit yang memberikan kontrol lebih besar terhadap proses animasi. modul-praktikum-flutter-pertemu…

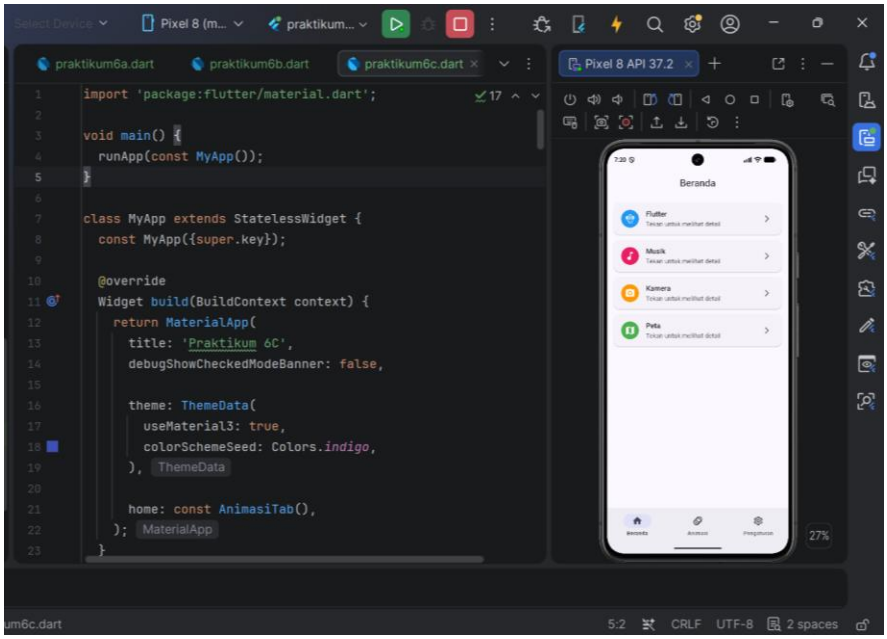


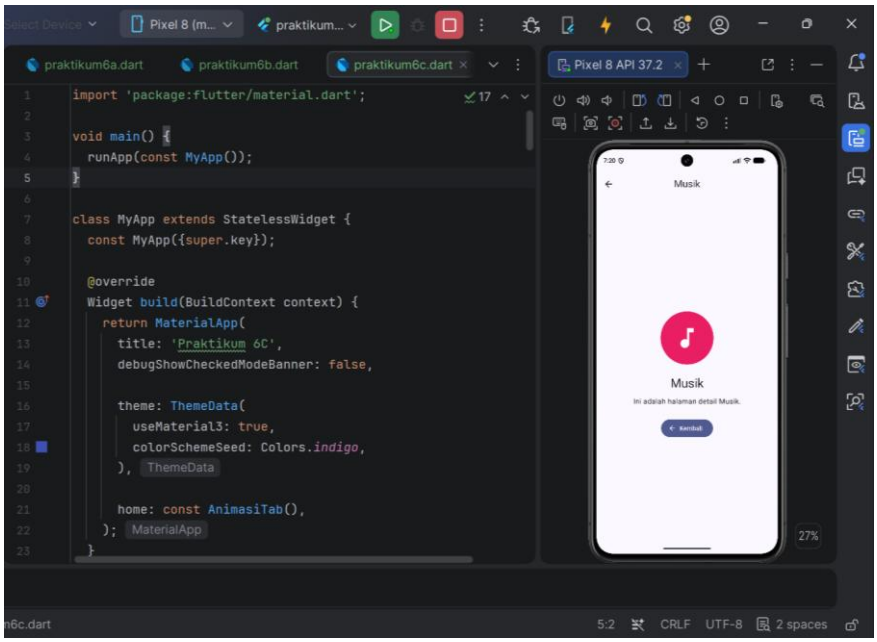


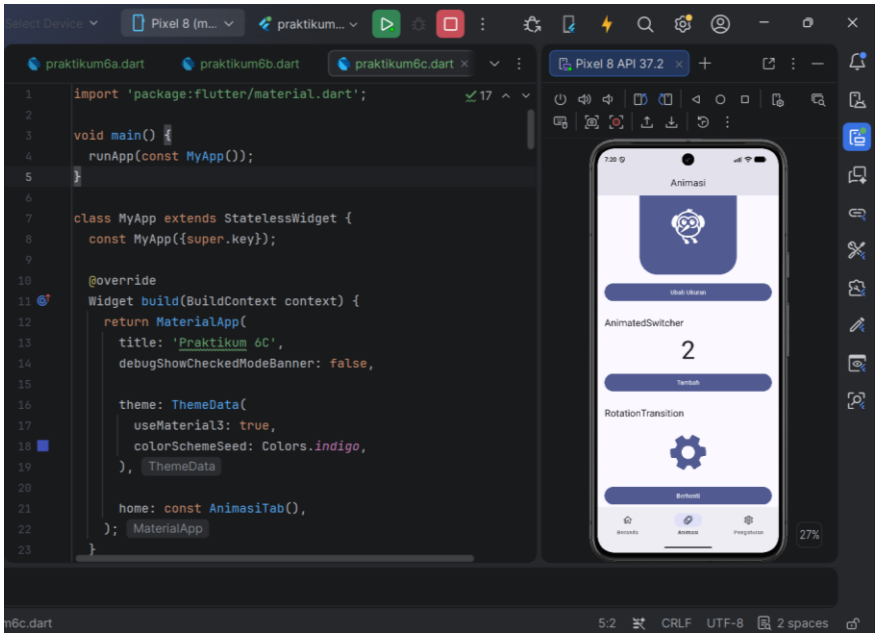
Bagian D — Navigasi Tab Bawah

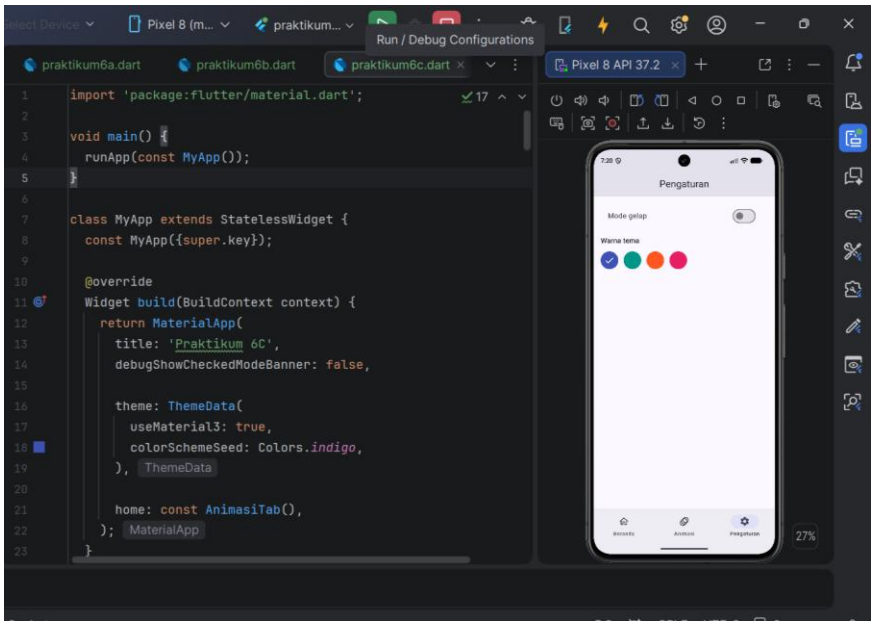
Pada Bagian D dilakukan penggabungan fitur-fitur yang telah dibuat pada bagian sebelumnya menggunakan NavigationBar dan IndexedStack. Aplikasi memiliki tiga tab utama yaitu

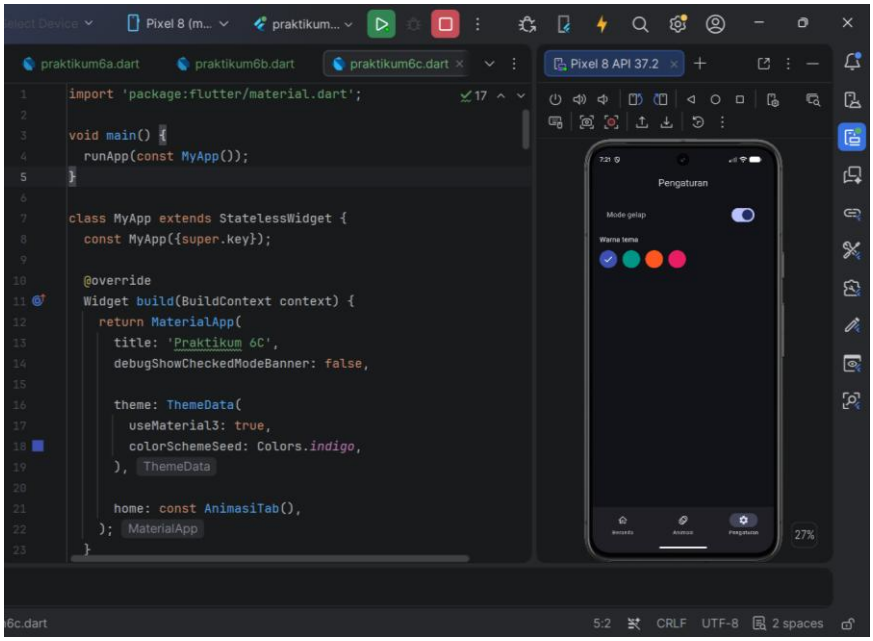
Beranda, Animasi, dan Pengaturan. NavigationBar digunakan sebagai navigasi tab bawah, sedangkan IndexedStack digunakan untuk menampilkan halaman berdasarkan tab yang dipilih sekaligus mempertahankan state setiap tab ketika pengguna berpindah halaman. Tab Beranda berisi daftar item dan navigasi menuju halaman detail dengan Hero, tab Animasi berisi beberapa contoh animasi, sedangkan tab Pengaturan digunakan untuk mengatur tema aplikasi. Dengan demikian, seluruh fitur tema, routing, Hero, dan animasi dapat digabungkan menjadi satu aplikasi Flutter yang terstruktur dan interaktif.






Pixel 8 (m...
praktikum...
praktikum6a.dart
praktikum6b.dart
praktikum6c.dart
Pixel 8 API 37.2
import 'package:flutter/material.dart';
void main() {
runApp(const MyApp());
}
class MyApp extends StatelessWidget {
const MyApp({super.key});
@override
Widget build(BuildContext context) {
return MaterialApp(
title: 'Praktikum 6C',
debugShowCheckedModeBanner: false,
theme: ThemeData(
useMaterial3: true,
colorSchemeSeed: Colors.indigo,
), ThemeData
home: const AnimasiTab(),
); MaterialApp
}
Animasi
AnimatedSwitcher
2
Tambah
RotationTransition
Berhenti
27%
5:2
CRLF
UTF-8
2 spaces

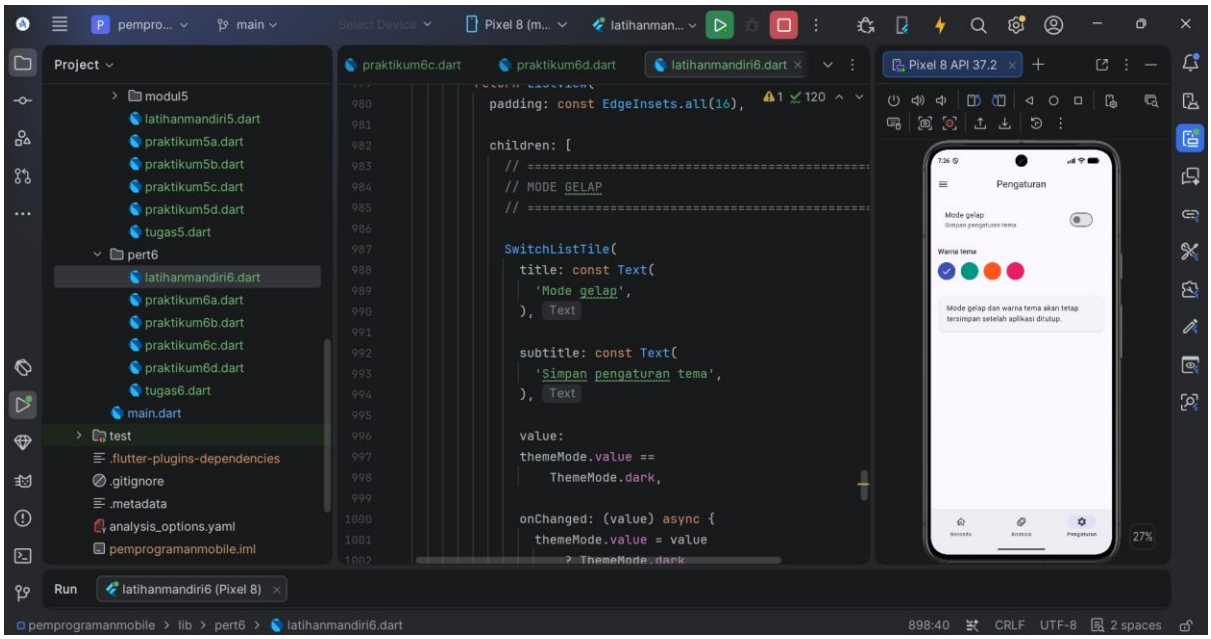

Run / Debug Configurations
Pixel 8 (m...
praktikum...
praktikum6a.dart
praktikum6b.dart
praktikum6c.dart
Pixel 8 API 37.2
import 'package:flutter/material.dart';
void main() {
runApp(const MyApp());
}
class MyApp extends StatelessWidget {
const MyApp({super.key});
@override
Widget build(BuildContext context) {
return MaterialApp(
title: 'Praktikum 6C',
debugShowCheckedModeBanner: false,
theme: ThemeData(
useMaterial3: true,
colorSchemeSeed: Colors.indigo,
), ThemeData
home: const AnimasiTab(),
); MaterialApp
}
Pengaturan
Mode gelap
Warna tema
27%

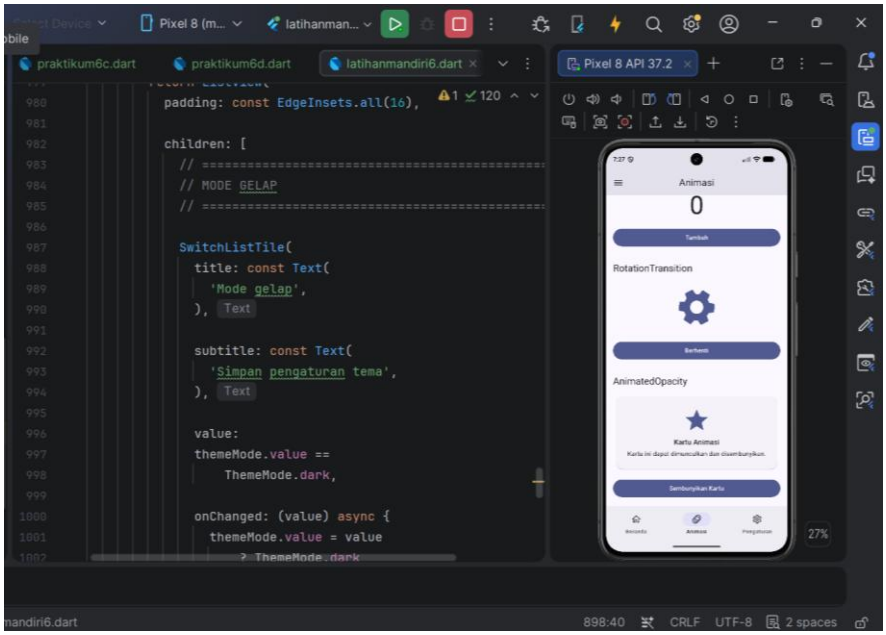


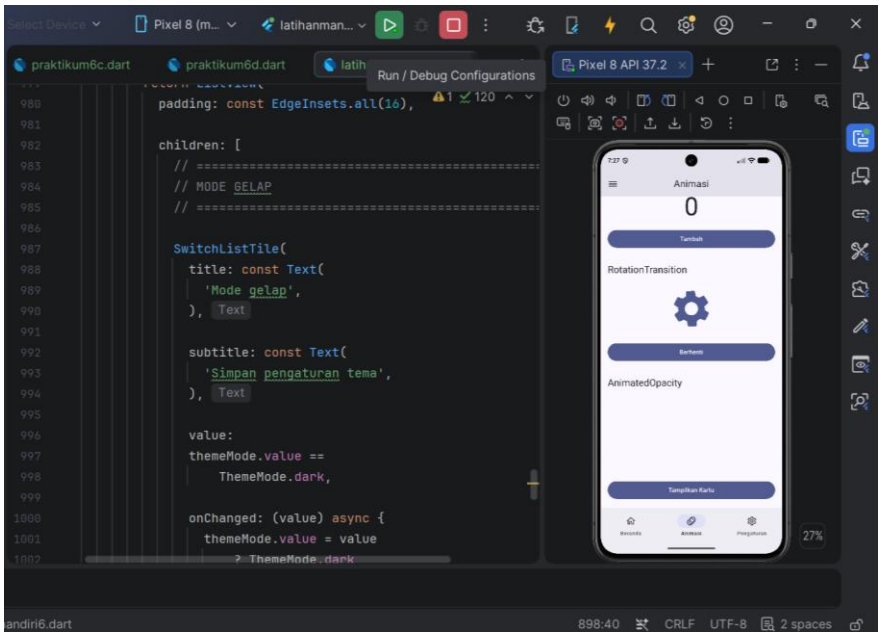
Latihan Mandiri

Pada latihan mandiri dilakukan pengembangan aplikasi dalam satu file main.dart dengan menambahkan beberapa fitur. Pengaturan mode gelap dan warna tema disimpan menggunakan shared_preferences, kemudian ditambahkan Drawer sebagai menu navigasi alternatif. Transisi halaman detail dikembangkan menggunakan PageRouteBuilder dan FadeTransition, sedangkan onUnknownRoute digunakan untuk menampilkan halaman 404 ketika route tidak ditemukan. Selain itu, ditambahkan AnimatedOpacity sebagai contoh animasi tambahan untuk membuat sebuah kartu dapat memudar dan muncul kembali.

Pengaturan mode gelap dan warna tema disimpan menggunakan shared_preferences sehingga pengaturan dapat dimuat kembali ketika aplikasi dijalankan.

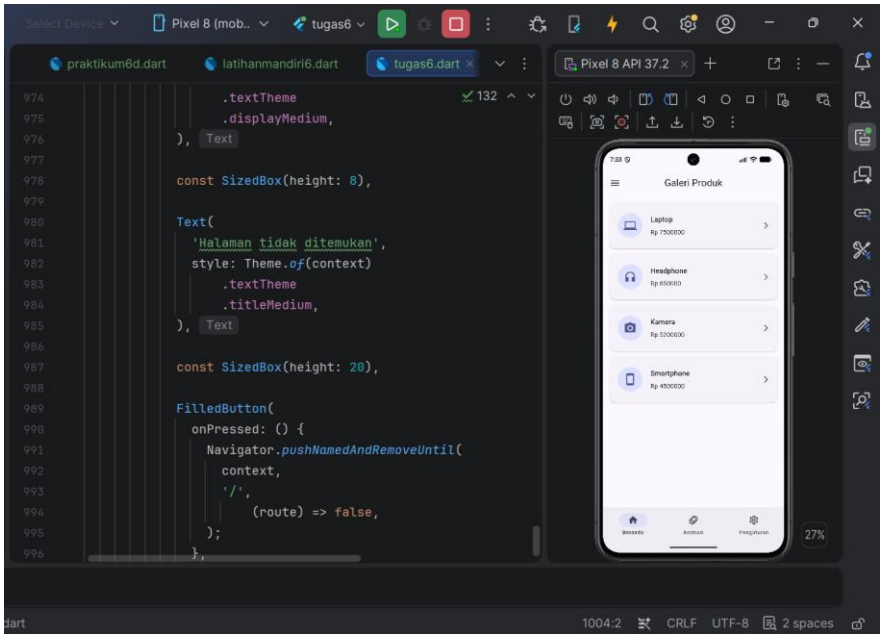


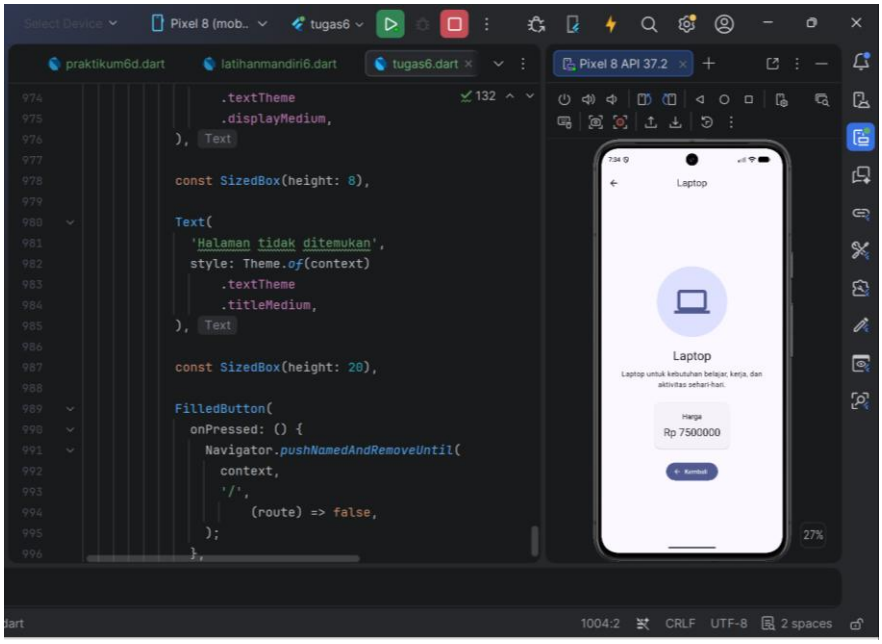


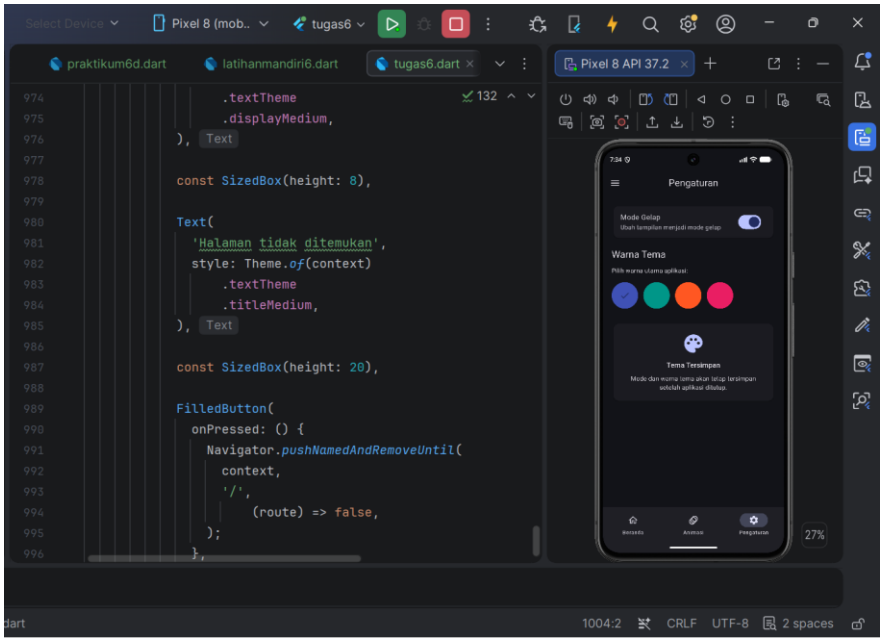


Tugas – Galeri/Katalog Interaktif

Pada tugas Pertemuan 6 dibuat aplikasi Galeri/Katalog Interaktif menggunakan Flutter dalam satu file main.dart. Aplikasi memiliki minimal tiga tab menggunakan NavigationBar, yaitu Beranda, Animasi, dan Pengaturan. Pada tab Beranda pengguna dapat memilih item untuk membuka halaman detail menggunakan named routes dan arguments, serta menggunakan Hero sebagai efek transisi. Pada tab Animasi diterapkan animasi implicit seperti AnimatedContainer dan animasi explicit menggunakan AnimationController. Pada tab Pengaturan pengguna dapat memilih mode terang atau gelap serta beberapa warna tema yang kemudian disimpan menggunakan shared_preferences. Warna dan gaya teks aplikasi menggunakan Theme.of(context) agar tampilan mengikuti tema yang dipilih dan tidak menggunakan warna hard-coded pada teks maupun latar.


praktikum6d.dart
latihanmandiri6.dart
tugas6.dart
Pixel 8 API 37.2
.textTheme
.displayMedium,
), Text
const SizedBox(height: 8),
Text(
'Halaman tidak ditemukan',
style: Theme.of(context)
.textTheme
.titleMedium,
), Text
const SizedBox(height: 20),
FilledButton(
onPressed: () {
Navigator.pushNamedAndRemoveUntil(
context,
'/',
(route) => false,
);
},
Galeri Produk
Laptop
Rp 7500000
Headphone
Rp 850000
Kamera
Rp 5300000
Smartphone
Rp 4500000
1004:2
CRLF
UTF-8
2 spaces


praktikum6d.dart
latihanmandiri6.dart
tugas6.dart
Pixel 8 API 37.2
.textTheme
.displayMedium,
), Text
const SizedBox(height: 8),
Text(
'Halaman tidak ditemukan',
style: Theme.of(context)
.textTheme
.titleMedium,
), Text
const SizedBox(height: 20),
FilledButton(
onPressed: () {
Navigator.pushNamedAndRemoveUntil(
context,
'/',
(route) => false,
);
},
Laptop
Laptop
Harga
Rp 7500000
1004:2
CRLF
UTF-8
2 spaces



Siappp, sekarang Pertanyaan Refleksi Pertemuan 6 yaa. Aku buat jawabannya naratif, jelas, dan langsung bisa dimasukkan ke laporan. Pertanyaannya sesuai modul ada 5.

Pertanyaan Refleksi

1. Apa perbedaan animasi implisit dan eksplisit? Kapan memilih masing-masing?

Animasi implisit merupakan animasi yang dijalankan secara otomatis oleh Flutter ketika terjadi perubahan nilai pada widget, contohnya AnimatedContainer dan AnimatedSwitcher. Jenis animasi ini lebih sederhana dan cocok digunakan ketika hanya membutuhkan perubahan tampilan yang tidak terlalu kompleks. Sedangkan animasi eksplisit menggunakan AnimationController sehingga jalannya animasi dapat dikontrol secara lebih langsung, seperti menjalankan, menghentikan, atau mengulang animasi. Animasi eksplisit dipilih ketika membutuhkan kontrol yang lebih besar terhadap proses animasi.

2. Mengapa dua Hero harus memiliki tag yang sama, dan apa yang terjadi bila tag-nya kembali muncul dua kali di halaman yang sama?

Dua widget Hero harus memiliki tag yang sama agar Flutter dapat mengenali bahwa kedua widget tersebut merupakan objek yang sama dan dapat melakukan transisi dari halaman asal ke halaman tujuan. Jika terdapat dua Hero dengan tag yang sama pada halaman yang sama, Flutter dapat mengalami konflik karena tag tersebut tidak lagi unik. Hal ini dapat menyebabkan error ketika aplikasi menjalankan transisi Hero.

3. Apa keuntungan memakai Theme.of(context) dibanding menulis warna langsung pada widget?

Penggunaan Theme.of(context) membuat widget mengambil warna dan gaya dari tema aplikasi yang sedang aktif. Keuntungannya adalah tampilan menjadi lebih konsisten dan dapat mengikuti perubahan tema, seperti ketika pengguna berpindah dari mode terang ke mode gelap

atau mengganti warna tema. Dengan cara ini, kita tidak perlu menuliskan warna secara langsung pada setiap widget sehingga kode lebih mudah dikelola

4. Apa perbedaan berpindah tab dengan IndexedStack dan membuka halaman dengan Navigator?

IndexedStack digunakan untuk berpindah di antara beberapa tab utama dalam satu halaman, seperti Beranda, Animasi, dan Pengaturan. Halaman-halaman tersebut tetap menjadi bagian dari struktur utama aplikasi. Sedangkan Navigator digunakan untuk membuka halaman baru di atas halaman sebelumnya, seperti ketika pengguna memilih sebuah item di Beranda kemudian membuka halaman Detail. Jadi, IndexedStack lebih cocok untuk navigasi antar-tab, sedangkan Navigator digunakan untuk perpindahan atau pembukaan halaman baru.

5. Mengapa AnimationController wajib di-dispose?

AnimationController harus di-dispose ketika widget tidak lagi digunakan agar sumber daya yang digunakan oleh controller dapat dilepaskan. Pada praktikum, controller dibuat ketika State diinisialisasi dan kemudian dihentikan serta dibersihkan pada dispose(). Hal ini penting agar controller tidak terus berjalan setelah halaman atau widget sudah tidak digunakan.

Kesimpulan

Berdasarkan praktikum Pertemuan 6, dapat disimpulkan bahwa Flutter menyediakan berbagai fitur untuk membuat aplikasi yang memiliki tampilan menarik dan navigasi yang terstruktur. Pada praktikum ini telah dipelajari penerapan tema terang dan gelap, perubahan warna tema, Theme.of(context), named routes, arguments, Hero, NavigationBar, dan IndexedStack. Selain itu, dipelajari juga penggunaan animasi implicit seperti AnimatedContainer dan AnimatedSwitcher, serta animasi explicit menggunakan AnimationController. Melalui latihan mandiri dan tugas Galeri/Katalog Interaktif, seluruh konsep tersebut dapat digabungkan dalam satu aplikasi, termasuk penyimpanan pengaturan tema, Drawer, FadeTransition, onUnknownRoute, dan AnimatedOpacity. Dengan demikian, praktikum ini memberikan pemahaman mengenai cara membangun aplikasi Flutter yang lebih interaktif, memiliki navigasi yang baik, serta tampilan yang dapat menyesuaikan pengaturan pengguna.